PRAGATI ENGINEERING COLLEGE(A)

ADSAA – ALGORITHM – VISUALIZER

PROJECT DOCUMENTATION:

SUBMITTED BY:

MD. ZAHURUNNISA BEGUM – 25A31A05EI

P. KEERTHANA – 25A31A05EN

J. MANIDEEP – 25A31A05FL

G. SUPRADEEP – 25A31A05FD

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ACADEMIC YEAR: 2026-27

ADSAA_ALGORITHM_VISUALIZER

1. ABSTRACT

The ADSAA Algorithm Visualizer is a web-based educational project designed to help students understand algorithms and data structures through visual representation. It provides learning modules for sorting algorithms, searching algorithms, graph algorithms, and data structures.

The project includes sorting algorithms such as Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort. It also covers Linear Search, Binary Search, Breadth-First Search (BFS), Depth-First Search (DFS), and Dijkstra's Shortest Path algorithm. The visualizer helps students understand the working process of algorithms step by step. It aims to make learning easier, more interactive, and more interesting.

2. INTRODUCTION

Algorithms and data structures are important concepts in computer science. They help solve computational problems efficiently. However, beginners may find it difficult to understand how algorithms work by studying only theoretical explanations or program code.

The ADSAA Algorithm Visualizer provides an interactive platform for learning these concepts. Students can explore different algorithm categories and observe algorithm operations through visualizations. The website also provides

ADSAA_ALGORITHM_VISUALIZER

sorting interface where users can enter numbers, select an algorithm, and control animation speed.

This project combines theoretical knowledge with practical learning and helps students improve their understanding of algorithmic logic.

3. PROBLEM STATEMENT

Learning algorithms and data structures can be challenging for beginners because their internal operations are not always easy to understand from written explanations or source code.

Students may struggle to understand comparisons, swaps, searching steps, graph traversal, and the differences between algorithms.

Therefore, a simple and interactive web-based system is needed to represent algorithms visually and help students understand their execution more effectively.

4. OBJECTIVES OF THE PROJECT

The main objectives of the ADSAA Algorithm Visualizer are:

- To provide an interactive platform for learning algorithms and data structures.

- To visualize sorting algorithms step by step.

- To help students understand searching techniques.

ADSAA_ALGORITHM_VISUALIZER

- To introduce graph algorithms and shortest-path concepts.

- To provide basic learning modules for stacks, queues, and trees.

- To display useful algorithm statistics, such as comparisons and swaps or moves.

- To improve students' problem-solving and logical-thinking skills.

- To make algorithm learning easier for beginners.

5. SCOPE OF THE PROJECT

The project focuses on providing a user-friendly educational platform for students studying Algorithms and Data Structures.

The system covers sorting, searching, graph algorithms, and basic data structures. Users can explore learning modules and experiment with numerical inputs in the sorting visualizer.

The project is mainly intended for educational purposes, classroom demonstrations, self-learning, and practical understanding of algorithm execution.

ADSA_ALGORITHM_VISUALIZER

6. EXISTING SYSTEM AND PROPOSED SYSTEM

Existing System

Traditional algorithm learning often depends on textbooks, classroom explanations, static diagrams, and manually tracing program code. These methods provide theoretical knowledge but may not clearly demonstrate every step of an algorithm.

Proposed System

The proposed system is a web-based algorithm visualizer that presents algorithm concepts through an interactive interface. It allows students to explore several algorithm categories and observe sorting operations with animation and statistics.

Comparison

The proposed system makes algorithm learning more interactive than relying only on static explanations. It allows students to experiment with inputs and observe algorithm behavior directly through the website.

ADSA_ALGORITHM_VISUALIZER

7. TECHNOLOGIES USED

The project website identifies the following technologies:

HTML: Used to structure the web pages and interface elements.

CSS: Used to style the website, including layouts, colors, and visual presentation.

JavaScript: Used to implement interactive behavior, user controls, and visualization functionality.

C++: Associated with the project's algorithm-learning context and programming concepts.

These technologies support the development of a web-based platform for interactive algorithm learning.

8. SYSTEM REQUIREMENTS

Hardware Requirements:

- Computer or laptop

- Keyboard and mouse or touchpad

- Sufficient memory to run a modern web browser

- Internet connection for accessing the hosted website

Software Requirements:

- Operating system: Windows, Linux, or macOS

- Web browser: Google Chrome, Microsoft Edge, or Mozilla Firefox

- Internet connection

- Access to the ADSAA Algorithm Visualizer website

9. SYSTEM ARCHITECTURE

ADSAA_ALGORITHM_VISUALIZER

The system follows a web-based architecture in which users interact with the application through a browser.

The user selects a learning module or enters numbers into the sorting visualizer. The application processes the selected options and displays the relevant visualization or learning content.

The main components are:

1. User Interface: Allows users to navigate learning modules and enter input.

2. Algorithm Module: Represents the selected algorithm's operations.

3. Visualization Module: Displays the algorithm's execution visually.

4. Statistics Module: Displays available information such as comparisons and swaps or moves.

5. Browser Environment: Runs the web application.

ADSAA_ALGORITHM_VISUALIZER

10. SYSTEM FLOWCHART

The general workflow of the application is shown below:

Start

↓

Open the ADSAA Algorithm Visualizer website

↓

Display the available learning modules

↓

Select a module

↓

Is the sorting visualizer selected?

- Yes: Select a sorting algorithm.

- Yes: Enter numbers.

- Yes: Set the animation speed.

- Yes: Start sorting and observe the visualization.

- No: Explore the selected searching, graph, or data structure module.

↓

Observe the available results or learning content

↓

End

ADSAA_ALGORITHM_VISUALIZER

11. MODULE DESCRIPTION

Sorting Algorithms Module:

This module introduces Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort. It helps students understand how elements are arranged in a particular order.

Searching Algorithms Module:

This module covers Linear Search and Binary Search. These algorithms help locate a target element in a collection of data.

Graph Algorithms Module:

This module introduces Breadth-First Search (BFS), Depth-First Search (DFS), and Dijkstra's Shortest Path algorithm. These algorithms are useful for understanding graph traversal and pathfinding.

Data Structures Module:

This module introduces stacks, queues, and trees. These structures are commonly used to organize and process data in computer programs.

Sorting Visualizer Module:

The sorting visualizer allows users to select a sorting algorithm, enter comma-separated numbers, and start the sorting process. Users can also adjust the animation speed.

Algorithm Statistics Module:

The interface displays algorithm statistics such as comparisons and swaps or moves. These values help students observe aspects of algorithm execution.

AUSAA_ALGORITHM_VISUALIZER

12. IMPLEMENTATION

The ADSAA Algorithm Visualizer is available as a hosted web application.

The implementation can be described through the following steps:

1. Open the project website in a web browser.

2. Explore the available algorithm learning modules.

3. Select the sorting visualizer when practicing sorting algorithms.

4. Choose an available sorting algorithm.

5. Enter numerical values separated by commas.

6. Use the input control to submit the numbers.

7. Adjust the animation speed as required.

8. Start the sorting process and observe the visualization.

9. Observe the displayed statistics and explore other learning modules.

The interface combines web page structure, styling, and interactive behavior to support algorithm learning.

ADSAA_ALGORITHM_VISUALIZER

13. TESTING AND RESULTS

Testing helps verify whether the website's main features behave as expected.

The following test cases can be used to evaluate the project:

Test Case 1: Website Access

Expected result: The website opens successfully in a supported browser.

Test Case 2: Module Navigation

Expected result: Users can select the available learning modules.

Test Case 3: Algorithm Selection

Expected result: The user can select an available sorting algorithm.

Test Case 4: Numerical Input

Expected result: The visualizer accepts appropriately formatted numerical input.

Test Case 5: Sorting Execution

Expected result: The selected sorting algorithm displays its sorting process.

Test Case 6: Animation Speed

Expected result: The speed control changes the animation speed when supported.

ASAA_ALGORITHM_VISUALIZER

Test Case 7: Statistics Display

Expected result: The interface displays the available comparison and swap or move statistics.

The results should be recorded after actually performing these tests on the website.

14. ADVANTAGES OF THE PROJECT

- Makes algorithm learning more interactive.
- Helps beginners understand algorithm execution.
- Provides multiple algorithm categories in one website.
- Allows users to experiment with numerical inputs.
- Provides animation speed control for sorting.
- Displays useful algorithm statistics.
- Supports self-learning and classroom demonstrations.
- Reduces dependence on static diagrams alone.

15. LIMITATIONS OF THE PROJECT

- Understanding the algorithms still requires basic programming knowledge.
- The visualizer covers only the algorithms and data structures implemented in the website.

ADSA_ALGORITHM_VISUALIZER

- Users need a compatible browser to access the hosted application.

- Internet access is generally required to open the hosted website.

- Visualizations alone may not provide complete knowledge of implementation details, proofs, or advanced complexity analysis.

- The availability and functionality of individual modules should be verified through practical testing.

16. FUTURE SCOPE

The project can be improved in several ways:

- Add more sorting and searching algorithms.

- Provide detailed explanations for every execution step.

- Include code examples in C, C++, Java, and Python.

- Add interactive visualizations for advanced data structures.

- Provide detailed time and space complexity comparisons.

- Allow users to compare two algorithms side by side.

- Add quizzes and practice exercises.

- Improve accessibility and mobile responsiveness.

- Provide a facility to save or download learning results.

ADSA_ALGORITHM_VISUALIZER

17. CONCLUSION

The ADSAA Algorithm Visualizer is an educational web application that helps students learn algorithms and data structures through interactive learning modules and visualizations.

The project brings together sorting, searching, graph algorithms, and basic data structures in one platform. Its sorting visualizer provides controls for selecting algorithms, entering numerical data, adjusting animation speed, and viewing algorithm statistics.

Overall, the project supports a more practical and engaging approach to learning computer science fundamentals. It can be further improved by adding more algorithms, explanations, coding examples, and assessment features.

ADSAA ALGORITHM VISUALIZER

18. REFERENCES

1. Textbooks on Algorithms and Data Structures.

2. Study materials related to Design and Analysis of Algorithms.

3. Concepts of sorting and searching algorithms.

4. Learning materials on graph algorithms and data structures.

5. HTML, CSS, and JavaScript study materials.

6. C++ programming concepts and algorithm implementation.

7. Classroom notes and reference materials provided by faculty.

19.ALGORITHM

Step 1: Start the program

Step 2: Open the Algorithm Visualizer website.

Step 3: Display the main menu containing Sorting Algorithms, Searching Algorithms, Graph Algorithms, and Data Structures.

Step 4: Select the required module and algorithm.

Step 5: Enter the required input, such as array elements, graph nodes and edges, or data structure values.

Step 6: Execute the selected algorithm or operation.

Step 7: Visualize the execution and display the result.

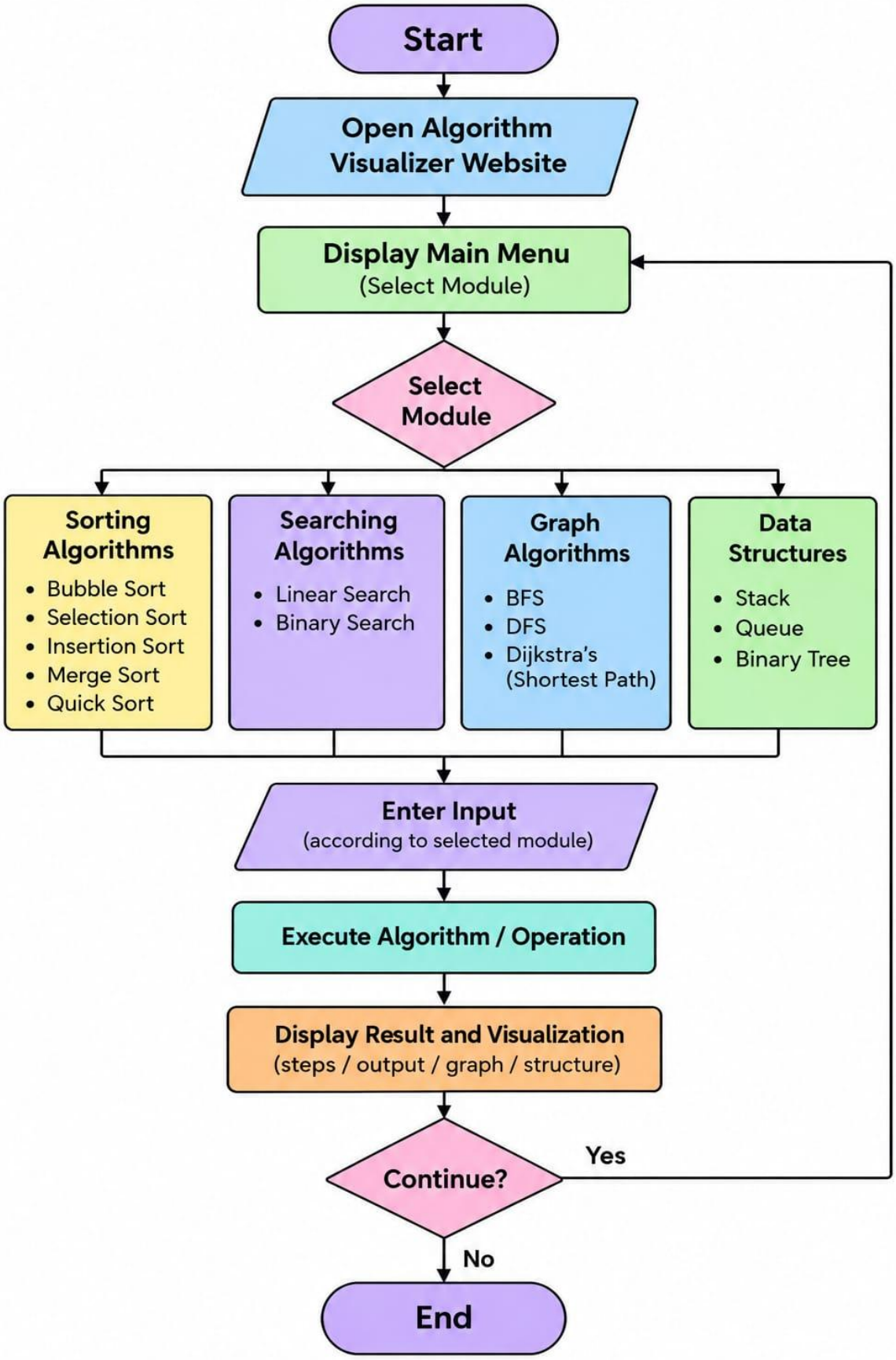
Step 8: Repeat the process if the user wants to try another algorithm or operation.

Step 9: Stop the program when the user chooses to exit.

Output: The website displays the algorithm's execution, results, and corresponding visualization.

ADSAA_ALGORITHM_VISUALIZER

20.FLOWCHART



ADSAA_ALGORITHM_VISUALIZER